UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

FACULTAD DE INGENIERIA DE PRODUCCION Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

Informe Avance Aplicativo Híbrido de Matrículas

Curso: Plataformas Emergentes

Docente: M. Sc. Ing. Rene Alonso Nieto Valencia

Elaborado por:
Mendoza Hilasaca, Emerson Danny

Arequipa 2026

Índice de contenido

1. Resumen..... 2
2. Descripción..... 3
3. Resultados obtenidos..... 4
4. Análisis de resultados..... 6
5. Conclusiones 7

Reporte

1. Resumen

El presente proyecto consiste en el desarrollo de un aplicativo híbrido móvil (Android/iOS) orientado a la gestión de matrículas universitarias. La aplicación permite a los estudiantes visualizar su malla curricular, matricularse en grupos de laboratorio disponibles y gestionar su perfil. Por otro lado, provee un módulo de administración que facilita la carga masiva de datos (estudiantes, cursos y grupos) mediante archivos Excel (.xlsx), automatizando la creación de registros en la base de datos local y proporcionando resúmenes estructurados del año académico.

2. Descripción

El aplicativo "App Matrículas" fue diseñado utilizando tecnologías modernas para el ecosistema móvil y web. El Frontend está construido sobre React Native con el framework Expo Router, garantizando una navegación fluida basada en pestañas (Tabs) y arquitectura MVVM (Model-View-ViewModel) para separar la lógica de negocio de la interfaz. Para los estilos se implementó Tailwind CSS (NativeWind), otorgando una interfaz limpia y atractiva. En el Backend, se utilizó Python con el framework Fast API, interactuando con una base de datos relacional local SQLite gestionada mediante SQLAlchemy. El sistema soluciona problemas nativos de permisos en dispositivos móviles (como Scoped Storage en Android) mediante la codificación de archivos a Base64 para una transmisión segura de documentos.

3. Funcionalidades

3.1. Funcionalidad 1: Carga Masiva de Datos (Administrador)

Permite al administrador poblar la base de datos subiendo archivos Excel. El sistema utiliza el motor nativo de JavaScript (FileReader) para convertir los archivos físicos a Base64 desde el dispositivo móvil, eludiendo restricciones de seguridad del sistema operativo. El backend procesa el Excel con la librería pandas, dividiendo inteligentemente los cursos y generando los correos institucionales automáticamente para los estudiantes nuevos.

3.2. Funcionalidad 2: Proceso de Matrícula (Estudiante)

Los estudiantes pueden acceder a una vista centralizada donde los cursos se agrupan por año académico. Al seleccionar un curso, el sistema despliega los grupos disponibles, sus horarios y la cantidad de vacantes en tiempo real. El estudiante puede inscribirse a un grupo con un solo toque, y el backend valida la disponibilidad y actualiza las vacantes de forma segura e inmediata.

3.3. Visualización de Cursos y Resúmenes

El sistema ofrece un Dashboard ("Resumen") para el administrador donde puede navegar por los años académicos y ver el estado de los cursos y sus grupos. De manera similar, el estudiante tiene la pestaña "Mis Cursos", donde puede visualizar el listado de las materias en las que se encuentra oficialmente inscrito.

4. Descripción del proceso de desarrollo

4.1. Arquitectura y Base de Datos

- **Almacenamiento:** La persistencia de datos se maneja a través de una base de datos SQLite embebida, la cual modela entidades relacionales (Estudiantes, Cursos, Grupos y Matrículas) mediante SQLAlchemy.
- Infraestructura Front-End: Se adoptó el patrón arquitectónico MVVM, agrupando la lógica de peticiones y manejo de estado en ViewModels (hooks personalizados) e inyectando los datos de forma reactiva en los componentes de la vista mediante Stores reactivos.

4.2. Comunicación (Frontend - Backend)

La comunicación se realiza mediante una API REST documentada automáticamente por FastAPI. Para mitigar bugs de red en componentes nativos (FormData sobre React Native en Android 13/14), las subidas de archivos fueron reestructuradas como peticiones POST en formato JSON, donde el binario del archivo viaja codificado en Base64. Esto garantiza una fiabilidad en la transferencia de datos.

4.3. Componentes Nativos de React Native y Organización del Layout

Para solventar el problema de adaptabilidad en diferentes resoluciones y garantizar una experiencia fluida, se emplearon componentes nativos de React Native estructurados bajo el modelo Flexbox:

- **View y Flexbox:** Estructura modular jerárquica con flexDirection (columnas y filas adaptables), gap y flexWrap para distribuir tarjetas de cursos y grupos sin desbordamientos.
- **ScrollView:** Manejo de desplazamientos verticales con rebote elástico nativo en listados de cursos y modales de detalle.
- **KeyboardAvoidingView con Platform.OS:** Organización del layout en la vista de login que ajusta dinámicamente la altura según el teclado del dispositivo para evitar oclusión de campos.
- **Expo Router (_layout.tsx):** Manejo de layouts persistentes mediante Tabs (navegación inferior aislada por roles) y Stack (navegación jerárquica con paso de parámetros dinámicos entre rutas).
- **ActivityIndicator y TouchableOpacity:** Feedback visual de estados asíncronos y retroalimentación táctil de alta respuesta.

5. Interfaces de la aplicación

5.1. Login

Pantalla inicial que permite el acceso diferenciado. El sistema distingue entre el rol Administrador (credenciales estáticas seguras) y rol Estudiante (correo institucional generado y CUI como contraseña).



Imagen. 01: Pantalla login

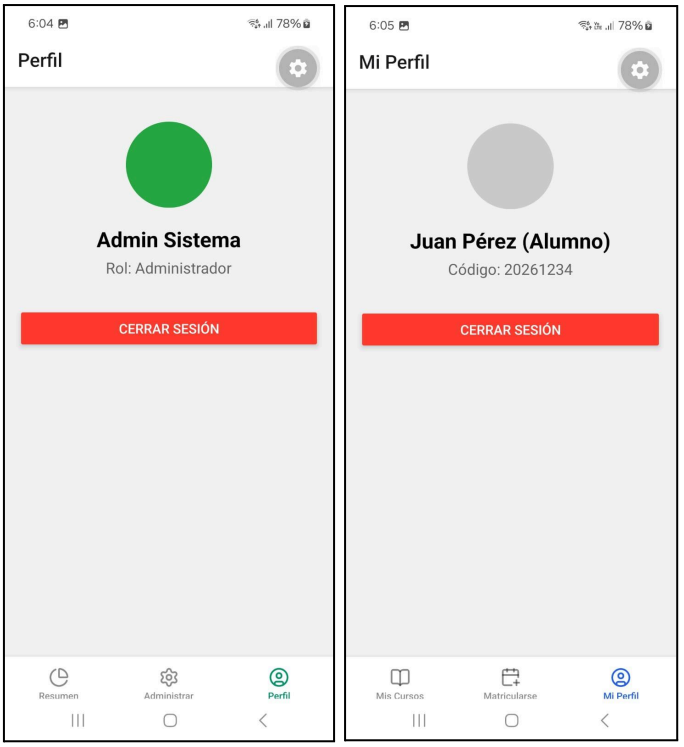


Imagen. 02: Roles de usuario

5.2. Administración

Pantalla que contiene los módulos operativos del Administrador, incluyendo las tarjetas para descargar plantillas y subir los datos de Cursos/Grupos y Estudiantes.

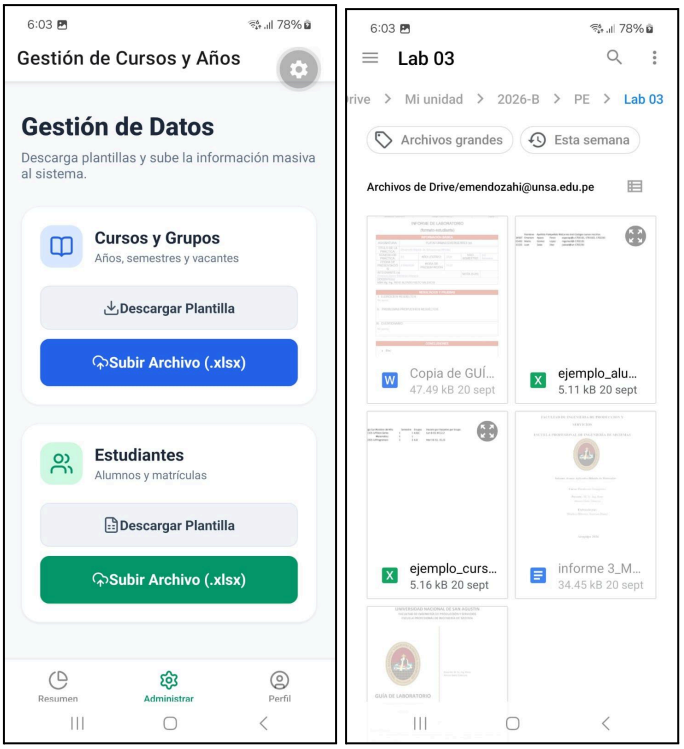


Imagen. 03: Interfaz de subida de datos

5.3. Perfil

Pantalla informativa que muestra los datos personales del usuario logueado (Nombres, DNI, CUI, Correo Institucional).

5.4. Resumen

Pantalla del Administrador que agrupa la malla curricular por año. Permite navegar hasta el detalle de cada curso y sus respectivos grupos.

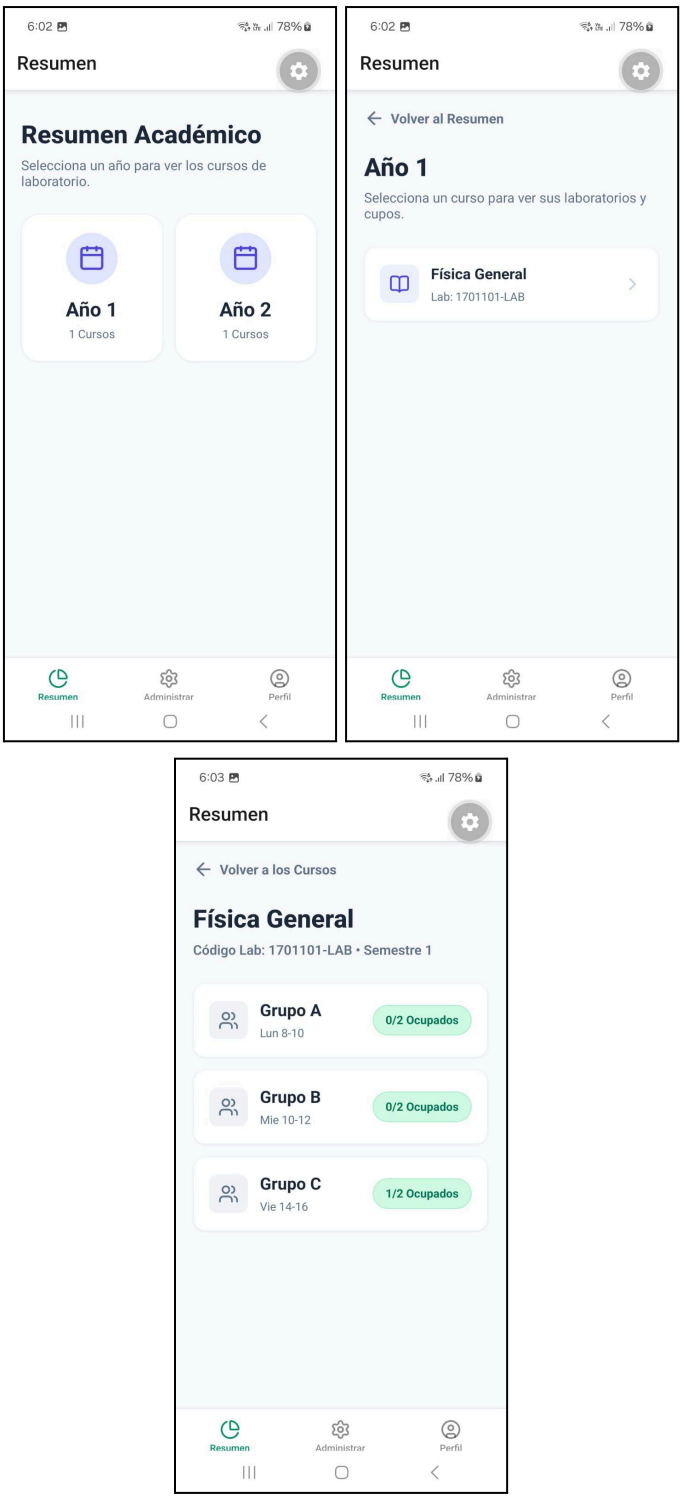
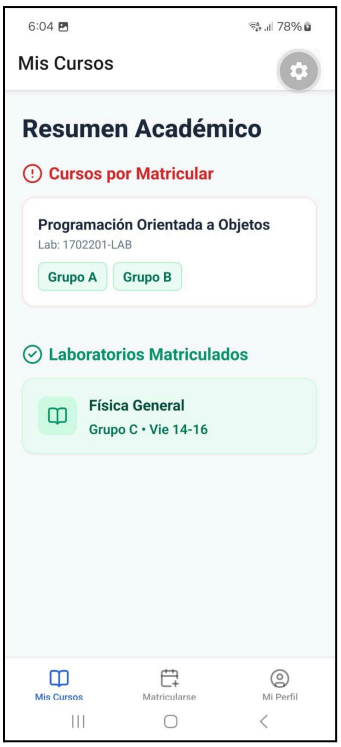


Imagen. 04: Interfaces del proceso de observación de matriculas

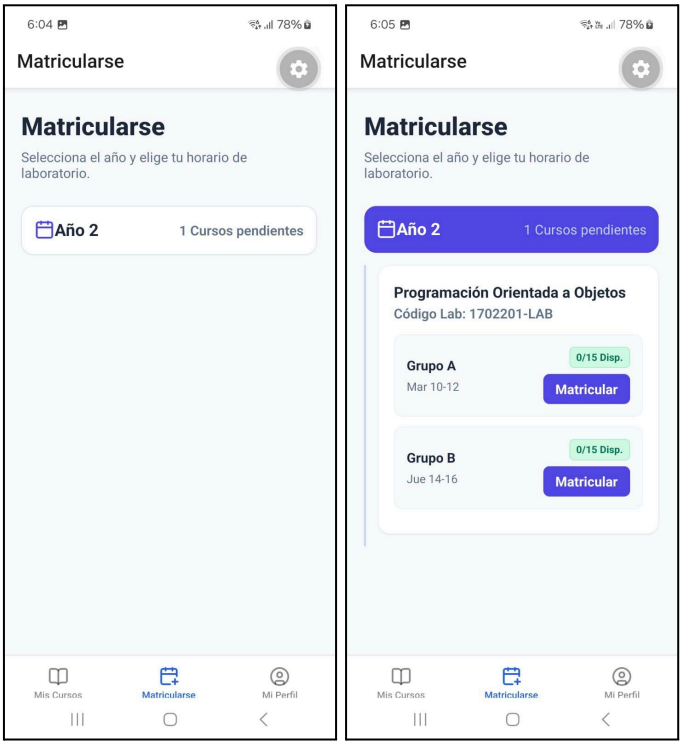
5.5. Matricularse

Interfaz del estudiante con diseño en acordeón o tarjetas que agrupa los cursos disponibles por año. Permite ver los laboratorios, horarios, vacantes restantes, y contiene el botón de acción para matricularse.



5.6. Mis matrículas

Pantalla principal del estudiante una vez logueado ("Mis Cursos"), donde se listan únicamente las materias en las que ya consolidó su inscripción. (Inserta aquí una captura de pantalla de la vista Mis Cursos del alumno)



5.7. Detalle de grupo y lista de alumnos inscritos

Resuelve la necesidad del administrador de auditar quiénes ocupan las vacantes de cada laboratorio. Al presionar un grupo desde el Resumen, se despliega una vista detallada con:

- Estado de cupos en tiempo real (capacidad total vs. ocupada).
- Tarjetas de cada alumno matriculado (Nombres completos, CUI, DNI y Correo Institucional).
- Herramientas administrativas de edición rápida (modificación de vacantes, cambio de horario o eliminación del grupo).

5.8. Auditoría y Prevención de Duplicados en Carga Masiva

Aborda la problemática de inconsistencia e inserción redundante de datos. El sistema implementa un algoritmo de validación previa (upsert) que inspecciona claves primarias y códigos únicos (ID de curso y CUI). Al subir un archivo Excel repetido o parcialmente nuevo, la interfaz notifica de inmediato el balance: registros nuevos creados vs. registros omitidos por duplicidad, conservando un registro histórico de auditoría.

5.9. Constancia oficial de matrícula en formato PDF

Da solución a la falta de comprobantes formales que acrediten la reserva de turno del alumno. Cuando el estudiante concluye su matrícula (o cuando las vacantes en cursos pendientes quedan agotadas), la vista inicial despliega una tarjeta de confirmación con el botón de descarga.
La app utiliza expo-file-system y expo-sharing vinculados al backend (ReportLab 5.0) para generar y abrir directamente en el celular una constancia universitaria con membrete formal, tabla de horarios asignados y código de verificación digital.

6. Lecciones aprendidas

- **Gestión de Permisos Nativos en Móviles:** Se aprendió que el ecosistema de React Native y Expo Router está fuertemente ligado a las políticas de seguridad subyacentes del SO (ej. Scoped Storage en Android). Modificar la aproximación clásica de FormData a una conversión local mediante FileReader y Base64 resulta ser una solución arquitectónica robusta ante errores de lectura de caché.
- **Patrón MVVM en React:** La separación de la lógica de negocio en ViewModels simplificó enormemente el desarrollo y limpieza del código UI, demostrando que patrones clásicos de software son sumamente efectivos en entornos modernos como React Native.
- **Manejo de Datos Masivos:** El uso de pandas en Python permitió estandarizar datos ruidosos o asimétricos provenientes de archivos Excel generados por usuarios, asegurando la integridad referencial en la base de datos relacional.
- **Generación nativa de documentos vs. WebViews:** En lugar de renderizar vistas HTML impredecibles entre diferentes versiones de Android, generar el documento binario directamente con librerías de vectorización (ReportLab) y transferirlo mediante el sistema

de archivos nativo (expo-file-system / expo-sharing) garantiza fidelidad de impresión y formalidad legal del documento.

- **Manejo de estados de bloqueo por capacidad:** Diseñar la interfaz considerando estados de contingencia (cursos saturados sin cupos disponibles) permitió adaptar dinámicamente el layout para que el flujo del estudiante concluya de manera limpia y no quede en un estado indefinido.

7. Anexos

- Enlace al repositorio de GitHub:https://github.com/MendozaEmerson/App-Matriculas

8. Referencias

- Documentación oficial de Expo y React Native: https://docs.expo.dev/
- Documentación de FastAPI: https://fastapi.tiangolo.com/
- Documentación de SQLAlchemy y SQLite.